PES UNIVERSITY

100 Feet Ring Road, Banashankari Stage

III, Dwaraka Nagar, Banashankari,

Bengaluru, Karnataka 560085

Dept. of Computer Science & Engineering

UE23CS342BA9: Generative AI and its Applications

Lab Exercise 4: Retrieval-Augmented Generation (RAG)

Course Anchor: Dr. Uma D

Teaching Assistants: Chirag A , Suchir M Velpanur

Exercise : Architecting Advanced Retrieval-Augmented Generation (RAG) Systems for Structured Screenplays

Objective

The goal of this exercise is to transition from basic semantic search to engineering a production-grade, context-aware advanced Retrieval-Augmented Generation (RAG) pipeline. You will move beyond naive text splitting and simple vector matching to implement layout-aware document parsing, hybrid search architectures, mathematical cross-encoder re-ranking, and dynamic LLM-as-a-judge evaluation frameworks.

By the end of the exercise, you should be able to:

- Understand and demonstrate the limitations of naive text chunking on highly structured spatial documents (like screenplays) and implement layout-aware parsing and semantic chunking to preserve critical scene boundaries, character alignments, and dialogue integrity.
- Initialize and benchmark embedding models and vector databases (such as FAISS or ChromaDB), analyzing the performance trade-offs of techniques such as approximate nearest neighbor indexing (HNSW) and Product Quantization (PQ) to optimize retrieval latency and scalable memory footprints.
- Establish a baseline Naive RAG chain by integrating a vector retriever with a Large Language Model using LangChain Expression Language (LCEL), applying strict system grounding prompts and conversational memory buffers.
- Implement Advanced Retrieval Enhancements by combining Dense Embeddings with BM25 Sparse Retrieval as a Hybrid Retrieval mechanism to capture both thematic intent and exact keyword matches (e.g., specific spells or scene numbers).
- Apply secondary mathematical re-ranking using Cross-Encoder models to re-score and refine retrieved candidate chunks based on deep contextual alignment rather than simple vector distance.
- Engineer dynamic query expansion modules that intercept and augment overly brief user prompts with lore-specific synonyms and expanded context before querying the vector database.
- Develop automated, ground-truth-free RAG evaluation suites by calculating dynamic Information Retrieval metrics such as Precision@K, Recall@K, NDCG@K, MRR and deploy TruLens as an LLM-as-a-judge framework to quantitatively benchmark pipeline hallucination and accuracy.

The Scenario

Modern interactive AI experiences and digital archives increasingly rely on Retrieval-Augmented Generation (RAG) to ground Large Language Models (LLMs) in factual, domain-specific documents. This architecture is the exact backbone powering enterprise knowledge assistants and modern search engines.

You are a Lead AI Architect at Leavesden Studios, the production powerhouse behind the Harry Potter film franchise. The studio is currently developing a massive, interactive 25th-Anniversary Digital Archive, allowing fans, VFX artists, and film historians to query the original production screenplay of *Harry Potter and the Sorcerer's Stone* to understand the director's original vision.

The engineering team rushed to deploy a standard Naive RAG pipeline using basic semantic embeddings and naive text chunking to search the screenplay script.

It was a disaster. During a pre-launch demo, the system failed critically in three ways:

1. Screenplays are highly structured, spatial documents. The naive text chunker blindly sliced through the script every 500 characters, frequently severing a Character's Name from their actual Dialogue block. When the RAG pipeline retrieved this orphaned dialogue, the AI had no idea who was speaking, confidently stating that Hagrid was giving Dumbledore's emotional speech about the Mirror of Erised.
2. When a production designer asked a thematic question, dense embeddings performed beautifully. However, when a continuity editor searched for a precise alphanumeric location tag like "Scene 165 - INT. TRAIN COMPARTMENT" or an exact fictional incantation like "Oculus Reparo," the semantic vector space failed completely. The dense embeddings blurred specific jargon into generic concepts of "trains" and "broken glass," retrieving entirely irrelevant scenes.
3. Compounding the issue, the naive brute-force similarity search (of the order of O(N)) began to severely lag when scaled up to simulate thousands of concurrent queries from fans, threatening the real-time interactivity needed for the exhibit.

To solve these critical data retrieval blind spots, you will investigate how advanced RAG techniques, such as layout-aware chunking, hybrid search, and mathematical re-ranking, can transform a naive LLM into a hyper-reliable, context-aware digital archivist.

The Task

You have been tasked with engineering a robust, production-grade RAG pipeline that can intelligently ingest highly structured PDF screenplays, bypass the limitations of dense vector embeddings, and autonomously evaluate its own hallucination rates without relying on hardcoded ground-truth datasets.

Instead of relying on basic API abstractions, you will engineer complex retrieval and generation architectures. You will build this system in four modules:

- **Module 1: Document Ingestion & Intelligent Chunking (Phases 1-2)** You will move beyond naive character splitting. You will implement layout-aware extraction (e.g. converting PDFs to markdown) and engineer a hybrid semantic chunking strategy that preserves crucial structural boundaries such as scenes, characters, and dialogue.
- **Module 2: Scalable Indexing & baseline RAG (Phases 3-4)** To solve the O(N) scalability issue, you will benchmark various vector database indexing algorithms (Flat L2, IVF, HNSW, and Product Quantization) to analyze latency versus accuracy trade-offs. You will then construct a baseline conversational RAG chain using LangChain Expression Language (LCEL), implementing conversational memory and custom routing logic to enforce strict anti-hallucination guardrails.
- **Module 3: Advanced Retrieval Enhancements (Phase 5)** To fix the semantic blurring of responses due to irrelevantly retrieved chunks, you will implement a hybrid search module, combining BM25 Sparse Retrieval (for exact keyword matching) with Dense Embeddings using the Reciprocal Rank Fusion (RRF) algorithm. You will then apply secondary mathematical re-ranking using Cross-Encoder models to re-score candidate chunks based on

deep contextual alignment, and engineer query expansion loops to proactively augment lazy user prompts.

- **Module 4: Exploratory Architectures, RAG Evaluation Metrics & LLM-as-a-Judge (Phases 6-7)** You will experiment with cutting-edge agentic workflows such as Hypothetical Document Embeddings (HyDE), RAG-Fusion, and Corrective RAG (CRAG). Finally, you will develop a dynamic evaluation suite using an LLM-as-a-Judge to calculate strict Information Retrieval metrics (Precision@K, Recall@K, NDCG@K, MRR) and deploy TruLens as an LLM-as-a-judge framework to quantitatively benchmark pipeline hallucination and accuracy.

Data Sources for Corpus and Dataset Schema

You will utilize a provided PDF file, screenplay.pdf, containing the original production screenplay of *Harry Potter and the Sorcerer's Stone*. This document is inherently spatial and highly structured, encompassing narrative elements such as exact scene headings (e.g., INT. GREAT HALL - NIGHT), character names, dialogue blocks, and action lines. You must use this corpus to construct a context-aware Retrieval-Augmented Generation (RAG) system capable of parsing these complex spatial boundaries to accurately retrieve both precise continuity details (e.g., spells, scene numbers) and synthesize broad thematic summaries without hallucinating outside lore.

Prerequisites

Technical Requirements

1. Python 3.10 or higher.
2. We strongly recommend running your labs on Google Colab and enabling a Hardware Accelerator (Runtime > Change runtime type > T4 GPU). Advanced RAG components, particularly local embedding models, cross-encoders, and local LLMs require significant compute and memory to execute without severe latency.
3. Unlike previous labs that relied on external cloud APIs with strict rate limits (e.g., Groq), you will be deploying a local Large Language Model directly inside your Colab environment using Ollama.
4. You are required to pull and experiment with open-weights models hosted on Ollama (e.g., llama3 or mistral) to act as the core generator for your RAG chains and the LLM-as-a-Judge for your TruLens evaluations.
5. You will also initialize open-source dense embedding models and cross-encoders hosted via HuggingFace (e.g., BAAI/bge-m3, nomic-embed-text, and BAAI/bge-reranker-base).
6. Required Packages**:** langchain_community, langchain_docling, langchain_text_splitters, langchain_huggingface, langchain_chroma, langchain_ollama, pypdf (or pymupdf), faiss-cpu, rank_bm25, sentence-transformers, trulens_eval.

Instructions Steps to Complete the Exercise

You will execute this lab in 7 distinct phases, moving from foundational document parsing, text chunking, vector indexing to engineering complex, advanced Retrieval-Augmented Generation (RAG) architectures and automated RAG evaluations.

Phase 1: Environment Setup & Data Extraction Before chunking and embedding, you must initialize the lab environment and extract the raw text from the highly structured Harry Potter screenplay PDF.

- Run the setup cells to install the necessary LangChain, HuggingFace, and other libraries
- You must implement **any two** distinct extraction methods. First, utilize standard PDF loaders (choose two from PyPDF, PyMuPDF, or PDFPlumber) to extract raw, unstructured text.
- Next, utilize DoclingLoader to extract the screenplay while preserving its spatial layout and converting it into structured markdown.
- Extract and print the first 3 pages of the document to verify your loaders are working correctly.

Phase 2: Chunking Strategies Naive chunking destroys the context of spatial documents. Your task is to intelligently divide the extracted screenplay into meaningful segments.

- For the natural text extraction, choose and implement two different chunking strategies (e.g., RecursiveCharacterTextSplitter, MarkdownHeaderTextSplitter). You must explain your choices in your final report.
- For the markdown extraction, implement a hybrid chunking strategy. First, use a MarkdownHeaderTextSplitter to split the text structurally by headers, and then pass those splits through a RecursiveCharacterTextSplitter to enforce a strict size limit.

Phase 3: Indexing, Embeddings, and Vector DBs You will translate your chunks into mathematical vectors and benchmark how different indexing algorithms handle search latency and accuracy.

- Initialize an open-source embedding model from HuggingFace (e.g., BAAI/bge-m3 or nomic-embed-text).
- Take your embedded chunks and build four distinct FAISS indexes: Flat Index (Brute-force), IVF Index (Clustered search), HNSW Index (Graph search), and PQ Index (Memory Compressed Search). Run the benchmark utility to compare their retrieval latencies.
- **Use 5 clusters for IVF Index, 16 edges per node for HNSW and 32 subquantizers, 8 bits per subvector for PQ**
- Embed your Docling Markdown chunks using the Nomic model and store them persistently in a Chroma vector database.

Phase 4: Baseline RAG, Memory & Branching Logic You will integrate your vector stores with a local Large Language Model using Ollama to establish your baseline generative pipeline.

- **Phase 4.1 (Retrieval & LLM):** Set up your local Ollama instance. Build a standard LCEL (LangChain Expression Language) chain that retrieves context, formats it, and passes it to the LLM via a structured system prompt. You could try prompting from a list of questions which are as follows (but not limited to):
    - What is the cause of Uncle Vernon's irritation?
    - How is Voldemort introduced and characterized?
    - How does the screenplay build suspense around the Sorcerer's Stone?
    - How is Hogwarts portrayed differently from the Muggle world?

- **Phase 4.2 (Memory):** Wrap your chain in RunnableWithMessageHistory to give the LLM conversational memory, allowing it to answer follow-up questions (e.g., asking *"What is written on it?"* after asking about Harry's cake). Some potential follow-up questions may look like:
    - Who gives Harry his first birthday cake? What is written on it?
    - What does Harry see in the mirror? And what does his friend see?
    - Tell me about the troll scene. What spell does Hermione use there? Who gets blamed for it afterward?
    - Why does Snape seem suspicious? Give examples. But who was actually responsible?
- **Phase 4.3 (Anti-Hallucination & Branching):** Write a strict system prompt to prevent the LLM from hallucinating movie lore not present in the text. Then, use RunnableBranch to route casual greetings to a simple chat chain, and factual questions to your strict RAG chain. Some interesting questions which you could try around with could be:
    - Who kills Snape in this movie?
    - When does Harry meet Sirius Black?
    - What does Dobby warn Harry about?
    - What happens in the Triwizard Tournament?

Phase 5: Advanced Retrieval Enhancements To fix the semantic blurring of dense embeddings, you will implement enterprise-grade retrieval techniques before the LLM even sees the prompt.

- **Phase 5.1 (Hybrid Search):** Initialize a BM25Retriever for sparse keyword matching. Write a custom retrieval function that executes both BM25 and FAISS searches, fusing their results together using the Reciprocal Rank Fusion (RRF) algorithm.
- **Phase 5.2 (Re-Ranking):** Initialize a Cross-Encoder model (BAAI/bge-reranker-base). Write a function that takes your Hybrid Search candidates and mathematically re-scores them based on deep query-document alignment, returning a refined top 3 documents.
- **Phase 5.3 (Query Expansion):** Users typically write lazy prompts. Build an LCEL chain that intercepts the user's query and expands it with synonyms and lore *before* searching the database.

Phase 6: Exploratory RAG Architectures You must select and implement **ANY TWO** of the following RAG workflows:

- **HyDE (Hypothetical Document Embeddings):** Force the LLM to write a fake, theoretical scene answering the query, and use that fake scene to search the vector database.
- **RAG-Fusion:** Generate 4 distinct variations of the user's query, search the database for all 4 simultaneously, and fuse the results using RRF.
- **CRAG (Corrective RAG):** Implement a self-reflection loop where a secondary auditor LLM grades the retrieved documents (Pass/Fail) and triggers a fallback mechanism if the context is irrelevant.

Phase 7: Automated Evaluation (LLM-as-a-Judge) You will evaluate your pipeline's accuracy dynamically without a hardcoded answer key/dataset.

- **Phase 7.1 (Dynamic IR Metrics):** Prompt your LLM to act as a strict binary relevance grader. Calculate the classical IR metrics (Precision@K, Recall@K, Mean Reciprocal Rank, and NDCG@K) based on the LLM's 1s and 0s.

- **Phase 7.2 (TruLens):** Deploy TruLens by writing 3 distinct grading prompts to calculate the context relevance, groundedness, and output relevance natively in Python, outputting the final scores in a pandas dataFrame.

IMPORTANT: Automated Evaluation Policy

READ CAREFULLY: This lab is graded using an automated evaluation script. To ensure you receive credit:

- DO NOT MODIFY any function signatures, names, or return types provided in the boilerplate code.
- DO NOT CHANGE the file structure or the names of the input/output files.
- DO NOT CHANGE any defined macros or model names that may be provided to you.
- STRICTLY FOLLOW the TODO blocks. You may add helper functions inside the block, but the main function definitions must remain exactly as given.

Failure to adhere to these constraints will cause the autograder to fail, resulting in a reduction of score assigned.

Submission Requirements

Your submission should include the following:

1. Code Implementation

- A Python file (<SRN>_GenAI_Lab4.py) containing all objectives implemented from scratch. (Download the Python file from Google Colab as: File > Download > Download .py)
- A Notebook file (<SRN>_GenAI_Lab4.ipynb) containing all objectives implemented with **clear, visible cell outputs.**

2. Report (<SRN>_GenAI_Lab4_Report.pdf)

A concise PDF report summarizing your observations and analysis. You must answer the following questions using trace outputs, scores, and specific evidence extracted from your Jupyter Notebook outputs:

1. Construct a comparative table documenting the final LLM-generated answers for the three provided test queries on **Baseline RAG** against the fully integrated **Advanced RAG** pipeline.

   Test queries:
   - "What is the cause of Uncle Vernon's irritation?"
   - “How does Harry's life change from beginning to end?”
   - Compare and contrast the way Harry Potter is perceived by his uncle and aunt versus wizards and witches around the world?

2. Provide a chunk output snippet from Phase 2 showing how naive character-splitting (RecursiveCharacterTextSplitter on raw text) caused context fragmentation(e.g., severing a character's name from their dialogue). Explain precisely how switching to DoclingLoader

Markdown extraction combined with MarkdownHeaderTextSplitter essentially preserves this critical [Character] -> [Dialogue] relationship.

3. Inspect your chunking parameters in Phase 2. If you set a chunk_size of 500 characters with an chunk_overlap of 0, what is the critical risk to semantic context? Conversely, if you set the overlap to 250, explain the impact this has on the total size of your FAISS index and the potential for redundant context retrieval.
4. Compare the retrieval latencies (in ms) of the Flat L2 and HNSW indexes from your Phase 3 benchmark logs. Explain the underlying graph-based traversal of HNSW. Why does Flat L2 exhibit O(N) time complexity while HNSW achieves logarithmic scaling, and at what cost to memory?
5. Analyze the Product Quantization (PQ) index performance. If PQ retrieved a highly irrelevant chunk for a specific query compared to the exact match found by Flat L2, explain the mathematical loss occurring during the vector compression phase. Why is PQ unsuitable to use for retrieving highly specific screenplay dialogue?
6. Examine your RunnableWithMessageHistory trace for the follow-up question as defined in phase 4.2. Explain how LangChain's memory buffer dynamically modifies the query vector before it hits the vector database.
7. Provide the output trace for the test queries executed in phase 4.3 and a casual "Hello" query. Explain the mechanism of RunnableBranch. Why is physically bypassing the vector database using conditional Python routing fundamentally safer, faster, and computationally cheaper than simply appending "Do not hallucinate" to a massive, single LLM system prompt?
8. Inspect your Phase 5.1 outputs for a highly specific keyword query (e.g., an exact incantation or Scene Number). Explain the phenomenon of semantic blurring inside the FAISS dense retriever. How did the sparse BM25Retriever solve this exact keyword match using term frequency?
9. In Phase 5.2, you passed candidate documents into BAAI/bge-reranker-base. Explain the architectural difference between how your initial bi-encoder generated embeddings independently versus how the Cross-encoder processes the (query, document) pair. If the Cross-encoder is vastly more accurate, why do we computationally restrict it to only re-ranking the Top-K candidates instead of searching the entire database?
10. Quote the exact expanded query generated by your LLM in Phase 5.3 for the lazy input: *"Harry is sad"*. How did appending lore-specific synonyms *prior* to the embedding step fundamentally shift the coordinates of the search vector to align with the screenplay's latent space?
11. Provide the snippet of the fake scene generated by your Hypothetical Document Embeddings (HyDE) chain. Explain the underlying mathematical paradox: How does intentionally forcing an LLM to hallucinate a fake document actually yield *better* nearest neighbors in a vector space than searching with the raw user question?
12. Provide the trace from the Corrective RAG (CRAG) pipeline where the auditor model evaluated a retrieved chunk and output no. Explain the exact mechanism by which this boolean gate intercepts the pipeline, triggers a fallback, and prevents "Garbage In, Garbage Out" (GIGO) hallucination loops.
13. Analyze your Phase 7.1 dynamic IR metrics. Assume your pipeline scored a Precision@3 of 0.33 but a Hit Rate (Recall@3) of 1.0. What does this mathematically indicate about the "noise ratio" being fed into your final generative LLM? Furthermore, explicitly explain why the NDCG@K formula applies a logarithmic discount penalty to relevant chunks retrieved at position 3 versus position 1.

14. Inspect your TruLens evaluation outputs. Identify or hypothesize a scenario where the groundedness metric scored exactly 0.0, but the Answer Relevance metric scored > 0.8. What exactly did the generative LLM do to achieve these contrasting scores? Explain why this specific combination is considered a threat when deploying RAG in enterprise environments.

Grading Criteria

Criterion	Weightage
Correct implementation	40%
Documentation and code readability	20%
Quality of analysis and discussion	30%
Presentation of results	10%